

Embedded Systems

Lecture 2

Paperduino → make your own processor

Choosing hardware:

- capability
- size
- price
- power cons.
- software environment

cheap, simple, single-thread

- MSP430
- AVR/ATMEL (Arduino)
- PIC

larger, 4 GByte mem, more capabilities

- ARM (Cortex)
- Quark (Intel)
- Freescale (out of style)

embedded system - class of system running on microcontroller-based hardware architecture

MCU → MARVEL → microcontroller unit

(Internet of Things (IoT) → connected to internet

Linker - tool that can compose an executable program

↳ takes compiled pieces and glues them together

↳ results in EF file (Executable & Linkable Format)

Hardware Interfaces Part 2 (video)

UART

NRF24601 (RF)

I²C

XBEE?

WIFI

SPI

Cellular Connection

SW architecture

arch1: Round-Robin → No interrupts required
One Event (polling)

```
while(1)
{
    if(event)
        handle-event();
}
```

arch2: Interrupts

Round robin w/ interrupts

```
while(1)
{
    if(flagA)
    {
        flagA = 0;
        handle-event A();
    }
}
```

```
ISR_A
{
    flagA = 1;
}
```

↳ issue: may share shared data

interrupt vector table → pointers to ISRs

arch3: OS Real-time - OS

RTOS → tasks (threads)

↳ designed for high priority
(unlike windows and others)

Interrupts

- real time - timing deadline

soft, firm, hard → how bad failure if missed
time

- interrupt pin asserted

- ↳ save inf. about current task

- ↳ program counter

- ↳ stack pointer

- ↳ register state

- ISR

- ↳ look at interrupt vector system

* No interrupt stacking! *

Shared Data Problem

ex:

```
static int iTemperatures[2];
```

```
void interrupt ReadTemperatures(void)
```

```
{ iTemperatures[0] = ...  
  iTemperatures[1] = ... }
```

```
void main(void) {
```

```
  int pTemp0, pTemp1;
```

```
  while (TRUE)
```

```
  {
```

```
    iTemp0 = iTemperatures[0];  
    " 1 = " " [1];
```

```
    if (pTemp0 != pTemp1)
```

```
      #ERROR!
```

Interrupt happens here?

itemp0 doesn't change!

what if use →

```
if (iTemperatures[0] != iTemperatures[1])
```

still same thing, because in assembly it still done separately.

fix → disableInterrupt(); enableInterrupt();

STM Timers (Video)

TM16, 15 general purpose timers

SystemCore → Timers → TM16 → channel 1

Input capture = store timer value

output compare = toggle timer

PWM = duty cycle

Prescaler 0 = prescaler of 1

Auto reload = max counting value (minus 1)

1/13 Computer arch notes (wrong notebook, whoops!)

Reading quiz

- 1.) Mother board
- 2.) T
- 3.) Memory, CPU, RAM, buses,
- 4.) Arnold, name? Jonas?
Carol? Aaron? Scott
- 5.) first microprocessor intel, 4004

Parallelism ^{for} CPU instruction-level / time

CPU = Processing Unit + Control Unit

→ ALU
→ Register file

→ Program Counter / Instruction Register
(ARM) (Intel) ^{Pointer}

memory → think of as one BIG long array

→ each address one byte

Word size → 32-bit → address $2^{32} = 4 \text{ GB}$

→ Kilo in computer → 1024 bytes

→ integer registers

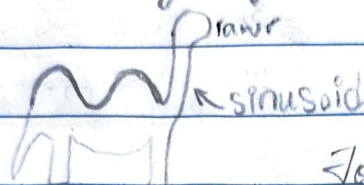
→ ALU size

MobaXterm

Signals Class (Who did I forget that notebook too?)

* shift then scale is easier!

Discrete signals can't have non-integer indexes!
(if transforming, they disappear)



$$\begin{cases} x(t) = e^{-2t} \cos(2\pi t) \\ x(t+T) = e^{-2(t+T)} \cos(2\pi(t+T)) \end{cases}$$

To determine periodic, replace t w/ $t+T$

$x(t) = x_1(t) + x_2(t)$, both periodic w/

$$T_1 = \frac{a}{b} \gamma \text{ and } T_2 = \frac{c}{d} \gamma$$

$$m' \frac{a}{b} \gamma = n' \frac{c}{d} \gamma$$

$$m'(ad) = n'(bc)$$

$$m' = bc$$

$$n' = ad$$

$$m = \frac{m'}{\text{GCD}(m', n')}$$

$$n = \frac{n'}{\text{GCD}(m', n')}$$

ex; $y(t) = 12 \cos(35t) + 10 \sin(49t)$

$$T_1 = \frac{2\pi}{35} ; T_2 = \frac{2\pi}{49}$$

$$T_0 = mT_1 = nT_2$$

$$\frac{m}{n} = \frac{T_2}{T_1} = \frac{\frac{2\pi}{49}}{\frac{2\pi}{35}} = \frac{35}{49} = \frac{5 \cdot 7}{7 \cdot 7} = \frac{5 \cdot 1}{7 \cdot 1} = \frac{5}{7}$$

$$T_0 = 5T_1 = 7T_2$$

$$T_0 = \frac{2\pi}{7} = \left(\frac{2\pi}{7} \right) \leftarrow \text{fundamental frequency}$$

$$y(t) = 12 \cos\left(\frac{10\pi}{3}t\right) + 10 \sin\left(\frac{5\pi}{4}t\right)$$

$$T_1 = \frac{2\pi}{\omega_1} = \frac{6\pi}{10\pi} = \frac{3}{5}; \quad T_2 = \frac{2\pi}{\omega_2} = \frac{8\pi}{5\pi} = \frac{8}{5}$$

$$T_0 = mT_1 = nT_2$$

$$= m\left(\frac{3}{5}\right) = n\left(\frac{8}{5}\right)$$

$$= 3m = n8$$

$$\frac{m}{n} = \frac{8}{3} \rightarrow m=8; n=3$$

$$T_0 = 8T_1 = 3T_2$$

$$= 8\left(\frac{3}{5}\right) = 3\left(\frac{8}{5}\right)$$

$$= \frac{24}{5} = \frac{24}{5}$$

* Even: $x(-t) = x(t)$

odd: $x(-t) = -x(t)$

$$\star \cos(\omega t) = \frac{e^{j\omega t} + e^{-j\omega t}}{2}; \quad \sin(\omega t) = \frac{e^{j\omega t} - e^{-j\omega t}}{2j}$$

$$\star e^{j\omega t} = \cos(\omega t) + j\sin(\omega t)$$

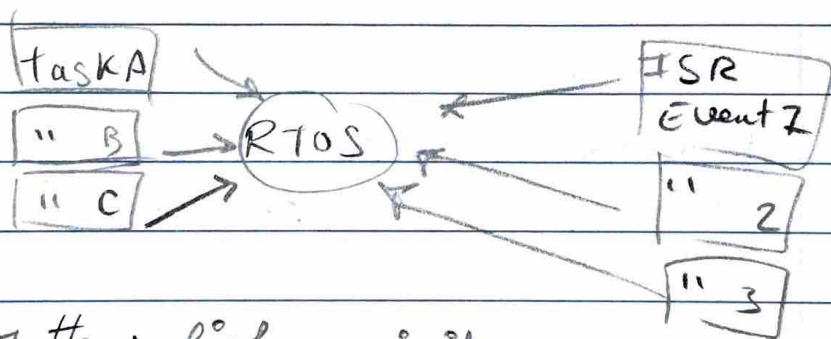
Study: Scheduler & Non-preemptive scheduler

RTOS

Kernel - core set essential programs

scheduler - scheduler what gets time on processor

- ↳ maps out tasks
- ↳ fairness algorithm (Windows)
(priority, not fairness)
- ↳ split work ISR & tasks



higher # → higher priority

RTOS → multitasking

↳ run forever w/out crashing

Preemptive RTOS → can pause/stop task
(save context)

↳ Register, stack, counter

non-preemptive scheduler, Lab assignment video

3 tasks:

- 1.) task run every second
- 2.) 1.5 second
- 3.) can run when other two aren't

* Not going to be exact

2.3 Integer Arithmetic

2.3.1 Unsigned Addition

x and y such that $0 \leq x, y < 2^w$

$$x + {}^u_w y = \begin{cases} x+y, & x+y < 2^w \text{ Normal} \\ x+y-2^w, & 2^w \leq x+y < 2^{w+1} \text{ Overflow} \end{cases}$$

Detecting overflow of unsigned addition

For x and y in range of $0 \leq x, y \leq UMax_w$,
let $s = x + {}^u_w y$. s overflowed if and only if
 $s < x$ (or $s < y$)

2.3.2 Two's complement Addition

For ints $-2^{w-1} \leq x, y \leq 2^{w-1}-1$:

$$x + {}^t_w y = \begin{cases} x+y-2^w, & 2^{w-1} \leq x+y \text{ Pos. overflow} \\ x+y, & -2^{w-1} \leq x+y < 2^{w-1} \text{ Normal} \\ x+y+2^w, & x+y < -2^{w-1} \text{ Neg overflow} \end{cases}$$

2.3.3 Two's Complement Negation

$$- {}^t_w x = \begin{cases} TMin_w, & x = TMin_w \\ -x, & x > TMin_w \end{cases}$$

2.3.4 Unsigned Multiplication

$0 \leq x, y \leq UMax_w$:

$$x * {}^u_w y = (x \cdot y) \bmod 2^w$$

2.3.5 Two's-Complement Multiplication

$$TMin_w \leq x, y \leq TMax_w$$

$$x \star_w y = U2Tw((x \cdot y) \bmod 2^w)$$

int is_mult_ok(int x, int y)

int p = x * y;

return !x || p/x == y;

2.3.6 Multiplying by Constants

$$x \ll k \rightarrow 2x$$

2.3.7 Dividing by Powers of 2

$$x \gg k \rightarrow \frac{1}{2}x$$

Two's complement: $0 \leq k < w$

$$(x + \underbrace{(1 \ll k) - 1}_{2^k}) \ll k \rightarrow x/2^k$$

$$(x < 0 ? x + (1 \ll k) - 1 : x) \gg k \rightarrow x/2^k$$

Ch 10

Task Management:

- task - independent execution units
- task control block - stores task's state, stack pointer, CPU context for switching

Scheduler Implementations:

- ↳ select next task to run
- Supervisor call - controlled software interrupt used by tasks to request services
- Cooperative scheduling - tasks voluntarily yield CPU
- Time slicing - divide CPU time into fixed intervals
- Blocking task - paused until condition met

Real-time Scheduling:

- ↳ meeting time constraints, not fairness
- Worst-case execution time - max execution time under worst conditions

Synchronization:

- Semaphore - signaling that indicates events
- Mutex - a lock, only one task at a time can access a resource
- Mutual exclusion - guarantee that a resource can't be used by multiple tasks

4.4 Build System

↳ Collection of tools and rules that automate compiling, assembling, and linking into an executable

- Toolchain - Dev tools (compiler, assembler, linker, debugger)
- Cross-compiler - runs on host machine, but generates code for different target processor
- Linker - combines object files into single executable
- Makefile - config file that defines build system compiler and linker

4.5 Startup code and Linker Scripts

- ↳ initializing hardware
- Reset vector - fixed mem address for where CPU should reset
- Linker script - controls how code and data are placed in memory, w/ sections and regions
- Watchpoint - halts execution when memory location changes ^{specific}
- Firmware - non-volatile memory that controls embedded hardware
- Bootloader - initializes system and loads/installs firmware

FreeRTOS

- high quality
- portable
- FREE!
- scalable
- always preemptive
- No restriction # tasks & # priorities
- Semaphores?
- Interrupts supported
- Runs on ticks, can change speed

Tasks → each is infinite loop
↳ don't return

Task delay: put task to sleep
↳ go to block queue

Suspended: not considered task

CMSIS - Common Microcontroller
software Interface Standard
↳ embedded C code portable, reusable
readable

↳ set of standards (not one library)
↳ lightweight
↳ standardizes how to access:

- CPU registers
- NVIC interrupts
- SysTick timer
- Startup clock config

SystemCoreClock - global holding CPU freq

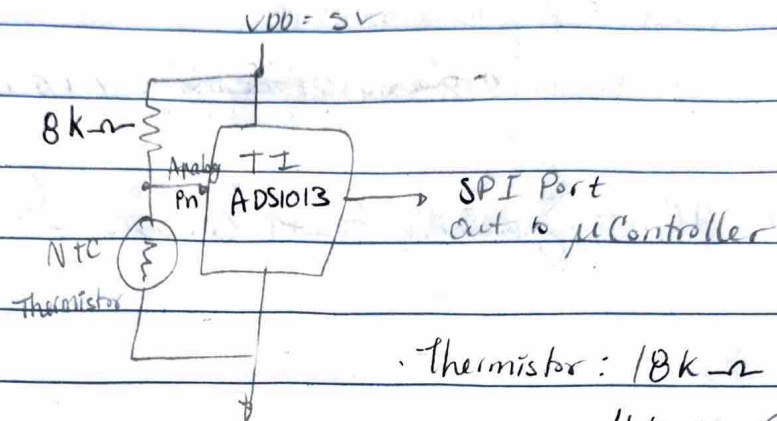
* CMSIS-NN (Machine Learning)

- ANN → pattern rec.
- RNN → time-series data
- CNN → image rec.

↳ integrated w/ tensorflow for micro.

Differential - more immune to noise
 ↳ two wires

Cellular: Arduino GSM shield (w/ SIM card - flat)



Thermistor: $18k\Omega$ @ 10°C
 $4k\Omega$ @ 50°C

$$5V \times \left(\frac{18k\Omega}{8k\Omega + 18k\Omega} \right) = 3.462V$$

$$5V \times \left(\frac{4k\Omega}{4k\Omega + 8k\Omega} \right) = 1.667V$$

$$\begin{array}{r} 3.462 \\ - 1.667 \\ \hline 1.794 \end{array}$$

$$5V / (12\text{bits}) \rightarrow 5V / 4095 = 0.00122V \\ = 1.22\text{mV}$$

$$\text{Slope} = \frac{\Delta V}{\Delta T} = \frac{1.794}{50-10} = 0.0449V/^\circ\text{C}$$

$$\frac{1.22\text{mV}}{44.9\text{mV}/^\circ\text{C}} = 0.027$$

Analog signals

sample twice as fast as max frequency

↳ Nyquist frequency

Aliasing - high frequency w/ not enough sampling results in a lower frequency

JTAG - Joint Test Action Group

Semaphores

Binary Semaphore & synchronization

↳ used to pace a task

Mutual exclusion - used to protect a shared resource

- kernel primitive object

two states: full (event occurred)

• empty (event not occurred)

→ shared resource, complete task before pass on key to another task

With Risks

Sem Take() · Sem Give()

Do NOT use mutual exclusive semaphore in an ISR!

- Higher priority task can "loan" its priority to a lower "as long as it is using a specific resource"

counting semaphore - how many resources are available, not specific resource

- semaphore - signaling mechanism used to coordinate tasks.
- Message passing - transfer data between tasks
 - ↳ usually w/ queues
- Queue has limited # of items
 - ↳ length - # of items
 - ↳ item size - size of each message
- Two ways to place data in a queue:
 - ↳ queue by copy
 - ↳ queue by reference
- Queue Set - structure that lets a task wait for data from several queues at once
- Critical section: must run w/o interruption
- Mutex (Mutual Exclusion): lock that allows only one task to access a resource
- Priority inversion: high priority task forced to wait for a lower-priority task
- Priority Inheritance: temp raise priority of task holding the mutex
- Deadlock: two tasks permanently waiting for each other
- Recursive Mutex - lock same resource multiple times
- Gatekeeper task: one task owns resource, another requests via queue

Ch 9 - Event Groups

- a task can wait for multiple events at once
- when an event happens, multiple tasks can wake up
- Event flag = 0 or 1 (event happened or not)
- Event bit set → all waiting tasks wake up
- Synchronization - tasks reach a point together

Ch 10 - Task Notifications

- task → directly notifies other tasks
 - ↳ no mutex, semaphore, or event group
- "direct-to-task" communication
- notification slot:
 - ↳ notification state (sending/not sending)
 - ↳ notification value (5 bytes)
- Task notifications = fast & less memory
 - ↳ not as flexible.
 - ↳ only one receiver
 - ↳ can't send data to ISR
 - ↳ no buffering w/ queues (only one message)

Lecture

- Pipe: dedicated connection between producer/consumer
- "Sockets": pipes w/ one or more endpoints that is a network connection

Just Ayrach Notes

⑦ Pressure sensor 0.98 mV/kPa , offset of 0.140 kPa at 0 V , 12-bit ADC w/ 2.5 V reference. ADC reads: $0x1AF$ ^(431₁₀)
What is the actual pressure? (kPa)

$$431/4095 \approx 0.1053 \times 2.5 \text{ V} = 0.263 \text{ V}$$

$$\begin{aligned} 0.263 \text{ V} / 0.98 \text{ mV} &= 268.5 \text{ kPa} + 0.140 \text{ kPa} \\ &= 268.636 \text{ kPa} \\ &\approx 269 \text{ kPa} \end{aligned}$$

⑧ max pressure?

$$2.5 \text{ V} / 0.98 \times 10^{-3} \text{ V} + 0.14 \approx 2551 \text{ kPa}$$

11. min bits for ADC to detect pressure changes as small as 0.25 kPa ?
w/ 0.98 mV/kPa sensitivity & 2.5 V reference

$$0.25 \text{ kPa} = \frac{\frac{1}{2^x - 1} \cdot 2.5 \text{ V}}{0.98 \times 10^{-3} \text{ V/kPa}}$$

$$\frac{(0.25)(0.98 \times 10^{-3})}{2.5} = \frac{1}{2^x - 1}$$

$$2^x - 1 = \frac{2.5}{(0.25)(0.98 \times 10^{-3})}$$

$$X = \left\lceil \log_2 \left(\frac{2.5}{(0.25)(0.98 \times 10^{-3})} + 1 \right) \right\rceil = 13.3$$

↳ 14 bits

Keyboard

"up", "down", "s", "d", "g"
"release"

ws://10.0.2.15:8080

tailscale VPN

RP

Camera sender.py

A/V packets

RP-STE-ROVER GUI-WINDOWS.PY

Speeds = [230, 240, 250]

Spins = [CW, CCW]

gear = [1, 2, 3]

Control data

WINDOWS computer

Sand

opcode,
motor,
1,
gppm,
reverse

ws://10.0.2.15:8080

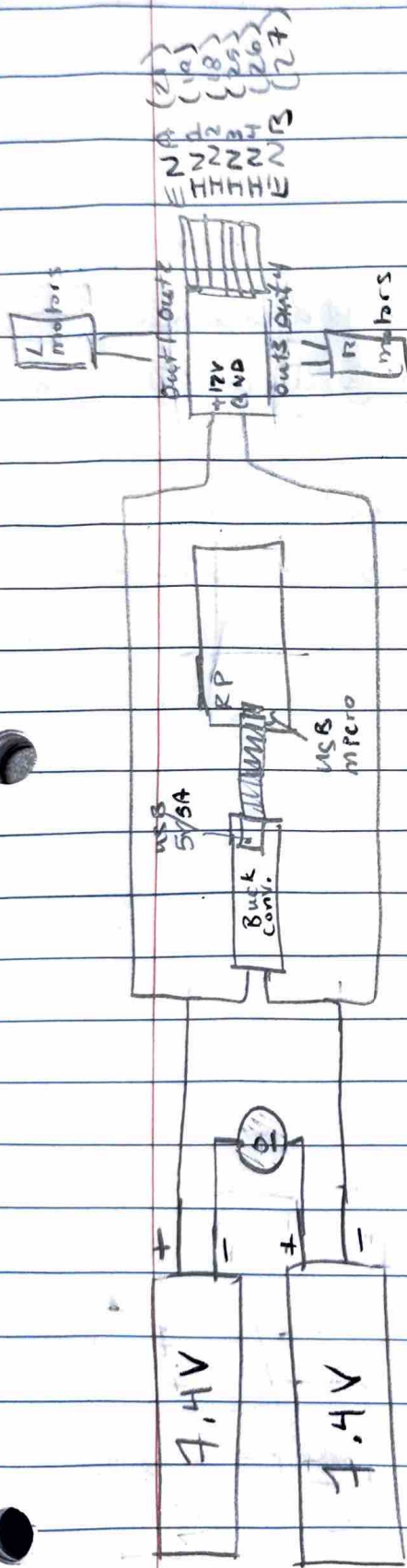
motor-control.py

0.75s timeout

pwm filter < 20

↳ duty cycle

GUI
Spin
Control
gear



Intro to IoT

↳ paradigm, combining technologies to create new services

Application Layer	Apps, A/V interfaces
Middleware Layer	Data processing
Network Layer	low data traffic. (WiFi, LTE, 5G, ...)
Sensing & Actuating Layer	Physical devices

Fog computing - data processing closer to IoT devices rather than data centers